

How AI Answers

In Vector Space, you watched a token's final vector land in its neighborhood on the map. That landing was about meaning: the model understanding the words you put in. Now we move to the next step: how AI answers you.

Start with your phone. As you type a text, it suggests the next word: three chips above the keyboard, picked from the last word or two, the same for everyone.

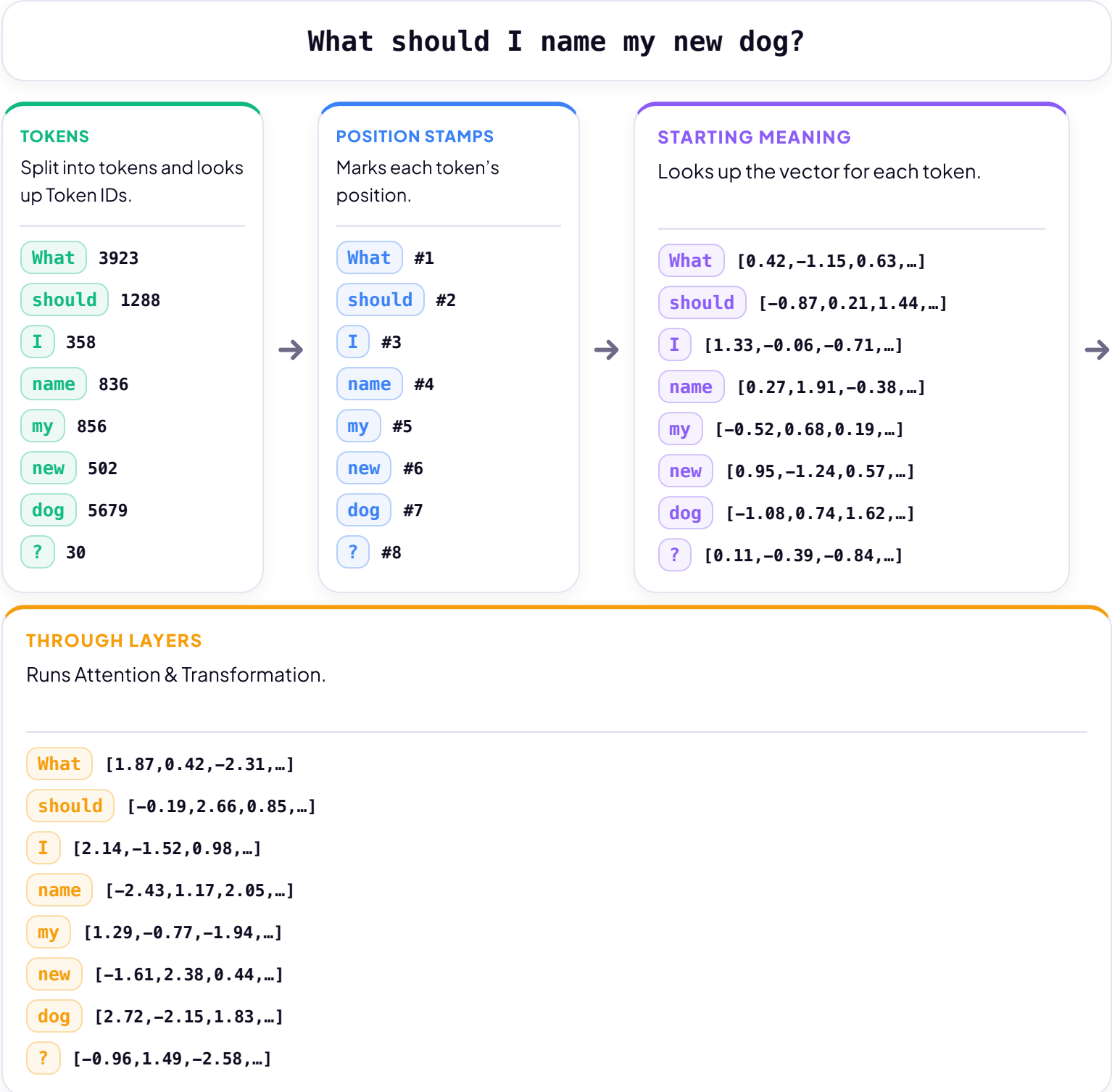
See you ____.

soon

tomorrow

later

AI follows the same basic logic as your phone's suggestion chips, but, as you might expect, it goes **way deeper**. Let's take a simple question and watch AI build its answer.



THROUGH LAYERS
Runs Attention & Transformation.

What

[1.87,0.42,-2.31,...]

should

[-0.19,2.66,0.85,...]

I

[2.14,-1.52,0.98,...]

name

[-2.43,1.17,2.05,...]

my

[1.29,-0.77,-1.94,...]

new

[-1.61,2.38,0.44,...]

dog

[2.72,-2.15,1.83,...]

?

[-0.96,1.49,-2.58,...]

WHERE WE STAND

Look at what the model has now. The question isn't words anymore. It's eight rich vectors, each one carrying what its token means inside this exact question. Meaning: established.

Now the detail everything turns on: **the last token matters most**. Attention doesn't just sharpen each token in place. Every pass also folds the earlier tokens into the later ones, so the last token's vector soaks up the entire question. By the final layer, it isn't really about the ? anymore. The layers have pushed that vector across the map, and it lands in the neighborhood of the words most likely to come next: the first word of the answer.

THE ANSWER, WORD BY WORD

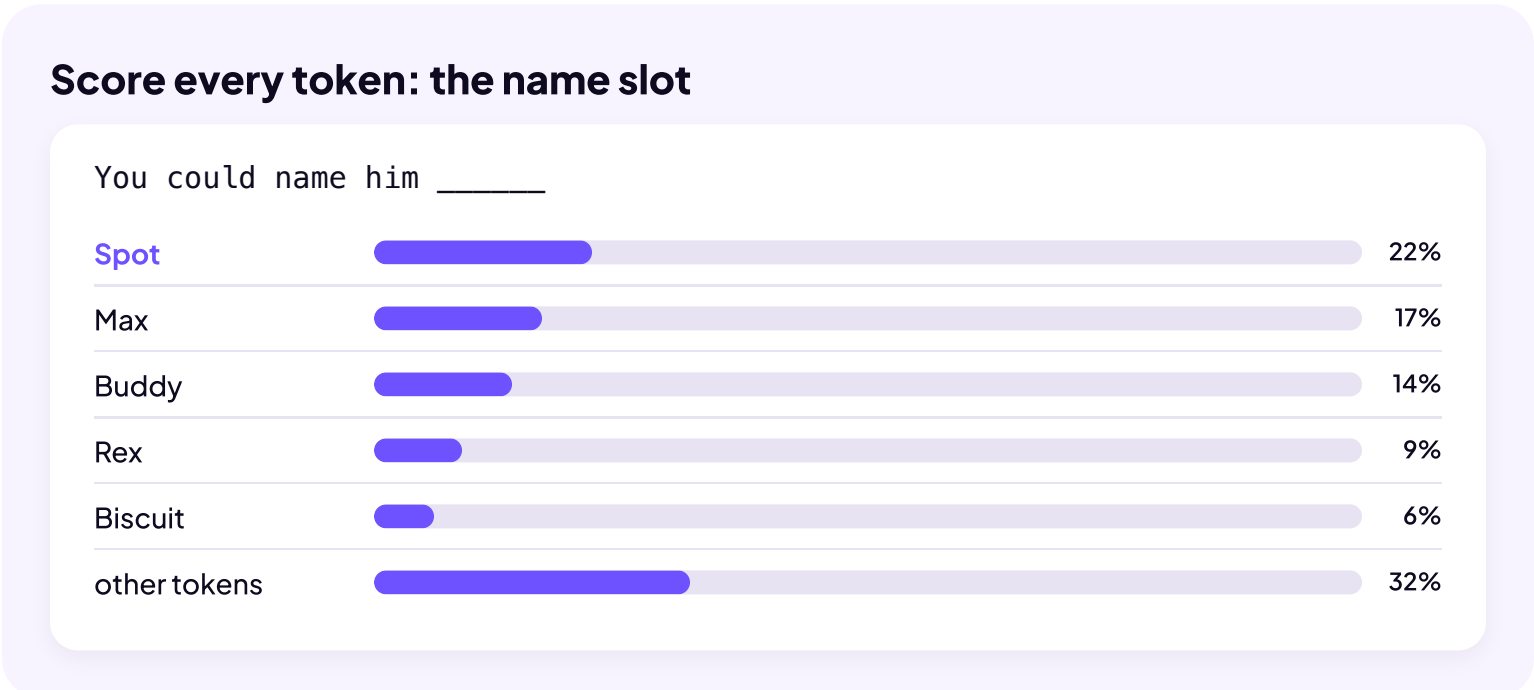
Now watch it happen. Each row below is one word of the answer. The model reads the vector sitting on the last token (the first column), checks which vector-space neighborhood it landed in, and takes the best-scoring word there. The pick gets typed, joins the context, and becomes the new last token. The next row runs the exact same move from there. That move is called **prediction**.

LAST TOKEN	REPLY SO FAR	NEIGHBORHOOD	TOP PREDICTIONS
?	_____	Reply starters	You 18% A 14% Great 9%
You	You _____	Ways to suggest	could 24% can 12% should 10%
could	You could _____	Naming verbs	name 31% call 19% try 7%
name	You could name _____	Who gets named	him 45% your 21% the 8%
him	You could name him ...	Dog names	Spot 22% Max 17% Buddy 14%
✓ You could name him Spot.			

Notice the name never came first. The neighborhoods tell the story: the sentence built its way, one slot at a time, to a spot where only a dog name fit. Every row is the same move; the only thing that changes is the context it starts from.

THE RANKED LIST

One more layer to be honest about: those three chips per row are just the top of the list. At every slot, the model scores every token it knows, thousands at once, and ranks them all. Here's step 5 under the microscope: the moment the name arrived.



The exact numbers are illustrative.

The model takes the top of the list and types it: Spot.

And that's the whole machine. Everything you just watched, from tokens to vectors to a ranked list to the next word, has a name: **inference**. It's what the model does every time it answers you.